

FinSave AI

Engineering Build Plan

From Concept Deck to Buildable MVP — Web-First, Hackathon Timeline

1. System Overview

FinSave AI is an AI-assisted financial coaching platform with four core capabilities: a spending tracker that ingests transactions, categorizes them, and generates summaries and alerts; a savings goal engine that recommends and tracks realistic savings plans; an MSME credit scoring engine that derives a score from cash flow and transaction behavior; and an insurance matcher that recommends products based on user profile.

The original deck describes what the product does. This document defines how to build it: architecture, data flow, technology stack, and a phased delivery plan that gets a working prototype live fast, then hardens it.

2. Guiding Constraints

Constraint	Decision
Platform	Web app first (mobile later, same backend/API)
Data ingestion	Bank API (Open Banking style) plus CSV upload — dual path
Scope	MVP / hackathon-ready — optimized for speed to demo, not scale

Because of this, the architecture is intentionally modular but lean: a single backend service, a Postgres database, a rules-plus-LLM hybrid for the AI components rather than training a model from scratch under a hackathon clock, and a clean React frontend.

3. Recommended Tech Stack — Craftsmanship-First, \$0 Cost

The stack below is chosen for two things at once: it costs \$0 to run at MVP/demo scale, and it signals to a technical reviewer that the build was done with care, not defaults. Familiarity with a tool is a fine tiebreaker, but it is not the primary criterion here — every choice below is justified on its own merits for a fintech prototype specifically.

Layer	Choice	Why it reads as professional	Free tier ceiling
Frontend	React + TypeScript + Vite + Tailwind CSS	Standard, fast, unremarkable in a good way — reviewers expect this and move on	Free (static hosting)
UI components	shadcn/ui + Recharts	Accessible, unstyled-by-default components read as deliberate design, not a template	Free (open source)
API layer	Fastify + Zod (or tRPC) instead of bare Express	Runtime-validated, typed request/response contracts — the single fastest way to signal correctness-mindedness in a finance app	Free (open source)
ORM / data access	Prisma on top of Postgres	A visible schema.prisma file and migrations read as production discipline; raw client calls scattered across routes read as a prototype cobbled together	Free (open source)

Layer	Choice	Why it reads as professional	Free tier ceiling
Database	PostgreSQL via Supabase Free	Real Postgres, not a toy DB — free tier gives 500 MB storage and unlimited API requests, enough for a full demo dataset	500 MB DB, 50k MAUs, 5 GB egress/month
Auth	Supabase Auth	Handles sessions, password hashing, and token refresh correctly out of the box — reviewers notice when auth is hand-rolled and broken	50,000 monthly active users free
Backend hosting	Google Cloud Run	Scales to zero, so it costs nothing when idle, and scales up cleanly under load during a live demo — this is what real production APIs run on, not a hobbyist PaaS	2M requests/month, 180,000 vCPU-seconds, 360,000 GiB-seconds/month, never expires
Frontend hosting	Vercel Free (Hobby)	Zero-config CI/CD from git push to live URL — expected baseline for a polished frontend deploy	Free for personal/small projects
Money handling	decimal.js or integer minor units, never raw floats	Using floats for RWF amounts is a known rookie mistake in fintech code review — this single choice is disproportionately noticed	Free (library)
File ingestion	papaparse / csv-parse	Handles malformed CSVs and encoding edge cases instead of a naive split-by-comma parser	Free (open source)
Bank API	Open Banking aggregator sandbox (e.g. Plaid sandbox)	Using a real aggregator's sandbox, even unpaid, shows you understand the actual integration pattern banks expect, not a fake mock	Free sandbox/dev tier
AI / Categorization	Rules engine + Claude API fallback for ambiguous cases	Deterministic-first design is both cheaper and more defensible to a lender than an all-LLM black box	Pay-per-token; keep volume low by using rules first
Credit scoring	Custom weighted formula, not an ML model	Explainable math beats an opaque model for a first version — you can show the exact weights on a slide	Free (your own code)
API documentation	OpenAPI spec, auto-generated from Zod schemas	A live, browsable API docs page during a pitch is a strong 'this is real software' signal with almost no extra effort	Free (open source)
Testing	Vitest on the scoring and categorization logic	A handful of tests on the credit-scoring function specifically tells a reviewer you don't trust untested code near a number that affects someone's loan	Free (open source)
Background jobs	node-cron inside the Cloud Run service, or Supabase Edge Functions	Cloud Run supports scheduled jobs via Cloud Scheduler on the free tier too, if you want them decoupled from the API process	Free within Cloud Run/Supabase quotas

3.1 What Changed From a "Just Get It Working" Stack

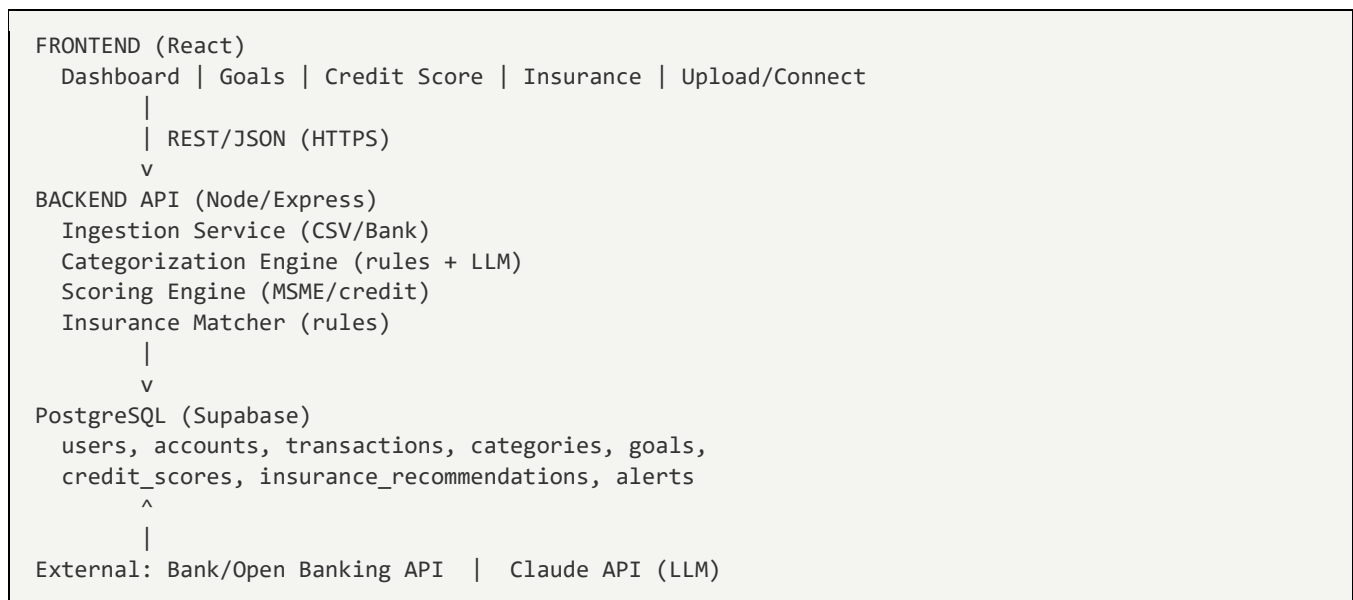
- Render/Railway replaced with Google Cloud Run — same zero-config deploy experience, but Cloud Run is what real companies run in production, and its free tier (2 million requests/month, scale-to-zero) comfortably covers a demo or even early real users.
- Bare Express swapped for Fastify + Zod — validates every request/response shape at runtime, not just at compile time, which matters when the data involves money.
- Raw Supabase client calls replaced with Prisma — gives you a single readable schema file and real migrations, which is what a technical judge will open first.

- Floats replaced with decimal-safe money handling — small change, disproportionately large signal in a fintech context.
- Testing and OpenAPI docs added even at MVP scope — both are near-zero effort with the tools above and both are the kind of detail that separates a weekend hack from a considered build.

3.2 One Cost Caveat

As of February 2026, Google Cloud requires a linked billing card to use Cloud Run even within the Always Free tier — this does not mean you get charged; it means Google wants a payment method on file before granting access to the free quota. Staying under 2 million requests, 180,000 vCPU-seconds, and 360,000 GiB-seconds per month keeps the bill at exactly \$0. For a hackathon demo this ceiling is not realistically reachable. Similarly, Supabase's free-tier projects pause automatically after 7 days with no API traffic — worth remembering if there's a gap between building and presenting; a single request to the project before a demo will wake it back up.

4. High-Level Architecture



5. Data Flow (End-to-End)

Step-by-step, matching the deck's "How the Entire Solution Works" slide:

- 1. Onboarding: user signs up via Supabase Auth and creates a profile (individual or MSME, income range, occupation/business type).
- 2. Data ingestion (two paths): Bank API path — user links account via Open Banking flow, backend pulls transaction history and normalizes it. CSV path — user uploads a bank/mobile-money statement, backend parses it with papaparse and normalizes into the same internal schema.
- 3. Categorization: each transaction runs through the rules engine first (keyword/merchant matching); anything unmatched goes to the LLM for classification, stored with category and confidence.
- 4. Spending summary generation: aggregation queries roll up transactions by category and month to power dashboard charts and generate alerts, e.g. spending exceeding a share of income.

- 5. Savings goal engine: user sets or accepts a suggested goal (e.g. a 600,000 RWF laptop within 12 months); backend computes required monthly savings, tracks contributions, and triggers reminders.
- 6. Credit scoring (MSME): scoring engine reads transaction history and business profile, computes a weighted score from 0 to 100, and stores an explainable breakdown.
- 7. Insurance matching: rules engine reads user profile (occupation, assets, dependents), maps to insurance product categories, and returns a ranked list with plain-language, LLM-generated explanations.
- 8. Dashboard: frontend queries all of the above via REST endpoints and renders a unified view.

6. Core Logic Design

6.1 Transaction Categorization

- Maintain a `category_rules` table mapping keywords to categories, e.g. MTN/AIRTEL to Airtime, SHELL/UBER to Transport.
- Unmatched transactions are batched and sent to the Claude API with a structured prompt that returns a category from a fixed enum.
- Confidence scores are stored; low-confidence classifications are flagged for user confirmation, which improves the rules table over time.

6.2 Savings Goal Recommendation

```
monthly_target = goal_amount / months_remaining
feasibility_check = monthly_target vs
    (avg_monthly_income - avg_monthly_essential_expenses)
if feasibility_check fails:
    suggest extended timeline or reduced goal amount
```

6.3 MSME Credit Score

A weighted formula — explainable, and no ML model required for the MVP:

Factor	Weight
Cash flow consistency (income variance)	30%
Transaction volume / frequency	20%
Repayment history (if any prior loans)	25%
Business age / stability	15%
Savings behavior	10%

The score is a weighted sum normalized to a 0–100 scale. The factor breakdown is stored so the UI can explain why a business received its score, matching the deck's "Credit Score: 85/100" example.

6.4 Insurance Matching

A simple rules table maps profile attributes to insurance products, as shown in the deck: a farmer maps to crop insurance, a shop owner to business insurance, a motorcycle rider to motorcycle insurance, and a family to health insurance. The LLM then generates a plain-language explanation of each recommended product and its premium range.

7. Database Schema — Core Tables

```
users (id, name, email, role[individual|msme],
      profile_json, created_at)
accounts (id, user_id, source[bank|csv], institution,
          last_synced_at)
transactions (id, account_id, date, amount, description,
              category, confidence, source)
categories (id, name, keywords[])
savings_goals (id, user_id, name, target_amount,
               target_date, monthly_target, current_saved)
credit_scores (id, user_id, score, factors_json,
               computed_at)
insurance_recommendations (id, user_id, product_type,
                           explanation, rank)
alerts (id, user_id, type, message, read, created_at)
```

8. Phased Delivery Plan (Hackathon Timeline)

Phase 0 — Setup (Day 1, few hours)

- Repo scaffold for frontend and backend, Supabase project, Vercel (frontend) and Cloud Run (backend) deploy pipelines.
- Auth wired end-to-end.

Phase 1 — Core Data Pipeline (Day 1–2)

- CSV upload and parser feeding the transactions table.
- Mock/sandbox bank API integration (e.g. Plaid sandbox) as the second ingestion path.
- Rules-based categorization engine.

Phase 2 — Spending Dashboard (Day 2)

- Aggregation endpoints for spending by category and month.
- Frontend dashboard with charts, alerts, and summaries — matches the deck's "Tracks Spending" slide.

Phase 3 — Savings Goals (Day 2–3)

- Goal creation, feasibility calculation, progress tracking, and reminder scheduling via node-cron.

Phase 4 — Credit Scoring (Day 3)

- Implement the weighted scoring function.
- Score breakdown UI — explainability matters for demo credibility.

Phase 5 — Insurance Matching (Day 3)

- Rules table plus LLM-generated explanations.
- Recommendation UI.

Phase 6 — LLM Integration Polish (Day 3–4)

- Claude API wired for ambiguous categorization, savings advice copy, and insurance explanations.
- Prompt templates finalized and tested.

Phase 7 — Demo Readiness (Day 4)

- Seed realistic demo data matching the deck's RWF examples.
- End-to-end walkthrough test: signup, upload, dashboard, goal, score, insurance.
- Deploy, polish UI, prep pitch.

9. Post-MVP Roadmap

- Real bank API partnerships, with per-market Open Banking compliance.
- ML-based credit scoring once labeled repayment data accumulates.
- Mobile app in React Native, reusing the same backend API.
- Multi-currency and multi-market support.
- SACCO/lender-facing API for credit score consumption, as noted in the deck: "Banks, SACCOs, or lenders can use this score."

10. Key Risks to Flag for the Team

- Bank API access: real bank integrations often require partnership and compliance approval — plan to demo with a sandbox or CSV path primarily.
- LLM cost and latency on categorization: keep the rules engine as the primary path, using the LLM only for the long tail.
- Credit scoring explainability: for lenders to trust it, the factor breakdown must stay transparent — avoid black-boxing this even later with ML.

11. Financial Institution Access — Lender Portal Design

A common misconception is that banks and SACCOs need access to the main FinSave AI system to use the credit scores. They should not. Giving lenders access to the full platform would expose every user's raw transaction data and personal financial information to a third party, creating serious privacy and liability risk. Instead, institutions get a separate, scoped interface: a different frontend, hitting a different set of API endpoints, with its own authentication and permission boundaries. They see scores and summaries, never raw transaction data, unless a specific user explicitly consents to share it for one loan application.

11.1 What Institutions Can and Cannot See

Data	Visible to lender?
MSME credit score (0–100)	Yes
Score factor breakdown (cash flow, repayment history, etc.)	Yes — aggregated, not line-by-line
Business category, age, region	Yes, if the business opts in
Raw transaction list	No — unless the user explicitly shares it for one application
Personal spending data (individual users)	No — institutions only see MSME profiles they evaluate, never individual consumer spending

11.2 Two Access Models

Model A — Ranked pool / marketplace. This matches the deck's line that banks, SACCOs, or lenders can use the score. Businesses opt in to being loan-ready and visible to partner lenders. Lenders log into their own dashboard and see a ranked, filterable list of opted-in businesses with scores. Clicking into a business reveals the score breakdown, never the underlying transactions.

Model B — Score lookup, bureau-style. A business applies for a loan directly with a specific bank. That bank calls FinSave AI's lender-facing API with a consent token tied to that one application. FinSave AI returns just the score and its breakdown for that one applicant. No standing list exists in this model — it behaves like a credit bureau check: request in, score out, nothing retained beyond the transaction.

Model B is more privacy-respecting and is the safer default; Model A is more useful for lenders who want to proactively source loan-ready businesses. Both can coexist — the opt-in pool feeds Model A, and any lender can still request a one-off score via Model B if the business grants consent.

11.3 Consent Is the Gatekeeper, Not Trust

No lender-facing endpoint should return data without an active, scoped, expiring consent grant. This is enforced with a dedicated table:

```
consent_grants (  
  id,  
  business_id,  
  lender_id,  
  scope      -- 'score_only' | 'full_profile'  
  granted_at,  
  expires_at,  
  revoked_at  
)
```

Every lender-facing API call checks for a matching, non-expired, non-revoked grant before returning anything. Businesses can revoke access at any time, and grants should auto-expire — for example after 30 to 90 days — rather than persisting indefinitely.

11.4 Architecture Addition

```
Lender Portal (separate app, separate auth)  
  |  
  v  
Lender-Facing API  
  GET /scores/:businessId (Model B)  
  GET /scores/ranked-pool (Model A)  
  |  
  v  
Consent/Permission Layer  
(checks consent_grants)  
  |  
  v  
credit_scores table  
(never raw transactions)
```


The lender portal is a completely separate frontend application from the consumer-facing FinSave AI app — different login, different users (bank staff, not consumers) — and it only ever talks to the lender-facing API surface, never the main app's internal endpoints.

11.5 New / Updated Database Tables

```
lenders (id, institution_name, contact_email,  
         api_key_hash, created_at)  
consent_grants (id, business_id, lender_id, scope,  
                granted_at, expires_at, revoked_at)  
loan_pool_opt_ins (id, business_id, active, opted_in_at)
```

11.6 Phase Addition to the Delivery Plan

Phase 8 — Lender Portal (post-MVP, or a Day 4 stretch goal if time allows):

- Lender authentication, separate from consumer auth.
- Consent grant table plus enforcement middleware.
- GET /scores/:businessId (Model B) and GET /scores/ranked-pool (Model A) endpoints.
- Minimal lender dashboard UI: list view plus score breakdown view.
- Opt-in toggle added to the business-side FinSave AI settings page.

For the hackathon demo itself, this can be represented as a lightweight read-only mock dashboard fed by the same credit_scores table, with the consent layer described but not fully enforced — real enforcement becomes the priority for the next iteration.

12. Demo & Sandbox Strategy — Making It Look Like a Real, Operating System

A convincing sandbox demo is not about faking data — it is about running the real pipeline against realistic, controlled data. Every layer, from ingestion through categorization, scoring, and recommendations, should be the actual production code path. The only thing that is sandboxed is the external data source at the very edge of the system. A demo built this way survives follow-up questions and live interaction; a demo built on hardcoded JSON does not.

12.1 Three Data Sources, Each Built to Look Real

Bank API path — Plaid Sandbox. Rather than hardcoding transactions, use Plaid's sandbox test users, such as user_transactions_dynamic or the persona-based user_small_business account, which ship with three months of realistic transaction history, including recurring transactions that appear at a natural pace rather than all at once. Demoing "connect your bank" goes through Plaid Link's real OAuth-style flow, lands on a real-looking bank login screen, and pulls real API responses. It looks identical to a production bank connection because structurally it is one.

CSV upload path — synthetic but statistically realistic statements. Generate CSVs matching real MTN Mobile Money or Bank of Kigali statement formats, with believable merchant names, RWF amounts, and natural irregularities — an occasional large one-off expense, a skipped month — rather than a suspiciously uniform dataset. This tells the story of an MSME with genuine cash flow noise, which is exactly what the credit scoring engine needs to look meaningful.

Live event injection — the highest-impact trick. Static, pre-loaded dashboards always look like screenshots, even when they are not. Using Plaid's /sandbox/transactions/create endpoint, a new transaction can be injected mid-demo,

which simulates a real refresh call and fires the same webhooks Plaid would fire in production. Triggering this live and showing the dashboard update, an alert fire, and the credit score recompute in front of an audience is the single most convincing proof that the system is actually operating, not just displaying preset numbers.

12.2 Demo Script — End to End

- 1. Onboarding: real Supabase signup, real session — no shortcuts here, since this is cheap to make genuine.
- 2. "Connect your bank": real Plaid Link sandbox flow — pick a sandbox institution, land back in-app with actually pulled data.
- 3. Dashboard populates from real aggregation queries against real (sandboxed) rows in Postgres, not a hardcoded JSON blob.
- 4. Trigger a new transaction via Plaid's sandbox API — a terminal command or an in-app demo button — and watch it appear, get categorized, and update the dashboard live, ideally without a page reload if polling or websockets are wired in.
- 5. Savings goal, credit score, and insurance match all read from that same underlying data, so the numbers stay internally consistent instead of the score saying one thing while the dashboard shows another.

12.3 Data Consistency — One Seeded User, One Coherent Story

Use a single, carefully seeded demo user across all four features rather than different random data per screen. If the credit score references strong repayment history, the transaction feed should visibly show those repayments. If a savings goal assumes a certain monthly surplus, the spending dashboard should show numbers that actually produce that surplus. Reviewers and judges cross-reference numbers across screens more often than expected, and any inconsistency is read as "this is fake" even when the underlying engine is real.

12.4 What This Costs

Nothing. Plaid's Sandbox environment is free and requires no production approval — it is designed specifically for this kind of development and demo use. The synthetic CSV generation is just a script. The live-injection trick uses the same free Sandbox API. This entire strategy fits inside the \$0 stack defined in Section 3 with no additional service or cost added.

13. API Endpoint Reference and Role-Based Access Control

This section is a planning reference, not a record of built code — this project has not been scaffolded yet, and no route names below are final. Treat every path here as a proposal to validate during Phase 0, not a contract already in place. The purpose of writing it out fully before any code exists is to catch access-control mistakes on paper, where they are nearly free to fix, rather than after they are wired into a real authentication system, where they are not.

Two separate API surfaces exist in this system and they must never share an authentication mechanism: the consumer-facing API, used by individual and MSME users through Supabase Auth sessions, and the lender-facing API, used by bank and SACCO staff through a completely separate login. A single shared login for both would be the single most damaging architectural mistake available in this design, since it would create a path for a lender credential to reach consumer data. Keep them structurally incapable of touching each other's data paths, not just logically separated by a permission check.

13.1 Roles in the System

Role	Who	Can access
individual	A consumer user tracking personal spending and savings	Only their own transactions, goals, and alerts
msme_owner	A business user tracking business finances	Only their own transactions, goals, credit score, and insurance matches; can opt their business into the lender pool
lender_staff	Bank/SACCO employee using the lender portal	Only score data for businesses with an active, non-expired consent_grant — never raw transactions, never consumer accounts
system/internal	Scheduled jobs (sync, scoring recompute, reminders)	Runs with its own service credential, not a user session — never exposed to a frontend

Every endpoint below states which role or roles may call it. Any endpoint not explicitly restricted to lender_staff or system must reject lender_staff tokens outright, and any endpoint not explicitly consumer-facing must reject individual/msme_owner tokens outright — the safe default is to deny and enumerate exceptions, not the reverse.

13.2 Consumer-Facing Endpoints — Your Backend

Authenticated via Supabase Auth session (individual or msme_owner roles).

Method / Path	Role required	Purpose
POST /api/v1/plaid/create-link-token	individual, msme_owner	Requests a short-lived token from Plaid so the frontend can open the Link widget. Never expose the Plaid client secret to the frontend — this call must happen server-side only.
POST /api/v1/plaid/exchange-token	individual, msme_owner	Receives the public_token after the user completes the bank connection flow, exchanges it server-side for a durable access_token, and stores it against the caller's own account_id — never a different user's.
POST /api/v1/plaid/sync-transactions	individual, msme_owner, system	Pulls new transactions for the authenticated user's linked account and writes them into the transactions table, scoped strictly to that user's account_id.
POST /api/v1/csv/upload	individual, msme_owner	Accepts an uploaded statement file, parses it, and normalizes rows into the same transactions schema as the bank path, scoped to the uploading user only.
GET /api/v1/transactions	individual, msme_owner	Lists the authenticated user's own transactions. Must filter by the authenticated user_id from the session, never by a client-supplied user_id parameter.
GET /api/v1/spending/summary	individual, msme_owner	Returns aggregated spend by category and month for the authenticated user only.

Method / Path	Role required	Purpose
POST /api/v1/transactions/:id/categorize	individual, msme_owner	Lets a user correct a transaction's category. Must verify the transaction belongs to the caller before allowing the update, not just that the caller is authenticated.
POST /api/v1/goals	individual, msme_owner	Creates a savings goal owned by the authenticated user.
GET /api/v1/goals	individual, msme_owner	Lists the authenticated user's own goals.
PATCH /api/v1/goals/:id	individual, msme_owner	Updates a goal. Must verify ownership before applying any change.
DELETE /api/v1/goals/:id	individual, msme_owner	Deletes a goal. Must verify ownership before deleting.
POST /api/v1/credit-score/compute	msme_owner, system	Triggers a score recalculation for the caller's own business. Not available to individual consumer accounts, since personal users are not scored.
GET /api/v1/credit-score	msme_owner	Returns the current score and factor breakdown for the caller's own business only.
GET /api/v1/insurance/recommendations	individual, msme_owner	Returns ranked insurance matches for the authenticated user's own profile.
GET /api/v1/alerts	individual, msme_owner	Returns the authenticated user's own unread alerts.
POST /api/v1/msme/loan-pool/opt-in	msme_owner	Lets a business owner opt their own business into the lender-visible ranked pool described in Section 11. This is the only endpoint that creates the possibility of lender visibility, and it must originate from the business owner, never be settable by a lender or by an internal job.

13.3 Lender-Facing Endpoints — Separate API Surface

Authenticated via a completely separate login path for bank/SACCO staff (lender_staff role). These must be served from routes, middleware, and — ideally — a distinct API subdomain that share no session logic with the consumer-facing endpoints above, so a bug in one surface cannot leak into the other.

Method / Path	Role required	Purpose
POST /api/v1/lender/auth/login	lender_staff (pre-auth)	Authenticates bank/SACCO staff through a login mechanism entirely separate from Supabase consumer auth.
GET /api/v1/lender/scores/:businessId	lender_staff	Model B score lookup for one specific business. Must check for an active, non-expired, non-revoked consent_grant matching this lender and this business before returning anything — return not-found rather than an error that reveals the business exists but is not consented, to

Method / Path	Role required	Purpose
		avoid leaking which businesses exist in the system.
GET /api/v1/lender/scores/ranked-pool	lender_staff	Model A — returns opted-in businesses only, filtered to loan_pool_opt_ins.active = true. Must never fall back to returning all businesses if the opt-in filter fails; fail closed, not open.
GET /api/v1/lender/consent-status/:businessId	lender_staff	Lets a lender check whether their consent grant for a business is still valid, without exposing any score or profile data if it is not.

13.4 Internal / System Endpoints

These are not called by any frontend. They run on a schedule (Cloud Scheduler or node-cron) using a service credential, never a user session token, and should not be reachable at all from the public internet if that can be avoided — prefer invoking them as internal Cloud Run jobs rather than exposed HTTP routes where the platform allows it.

Job	Trigger	Purpose
Transaction sync sweep	Scheduled, e.g. every few hours	Calls Plaid's sync endpoint for all linked accounts and writes new transactions
Credit score recompute	Scheduled, e.g. nightly, or on new transaction batches	Recalculates MSME scores so lender-facing lookups stay current
Savings goal reminders	Scheduled, e.g. daily	Sends reminder notifications for active goals
Consent grant expiry sweep	Scheduled, e.g. daily	Marks expired consent_grants as inactive so lender lookups fail closed automatically once a grant lapses

13.5 External Third-Party API Calls

These are calls your backend makes outward, not endpoints your backend exposes. They should only ever be called from server-side code — never directly from the frontend, since every one of them requires a secret credential.

Service	Endpoint	Called by / Purpose
Plaid	POST /link/token/create	Your backend, when a user clicks Connect Bank
Plaid	POST /item/public_token/exchange	Your backend, immediately after the user completes the Link flow
Plaid	POST /transactions/sync	Your backend, on-demand and via the scheduled sync sweep
Plaid (sandbox only)	POST /sandbox/transactions/create	Your backend, only in the sandbox environment, to power the live demo trick from Section 12 — this call must be disabled or unreachable outside sandbox/demo builds

Service	Endpoint	Called by / Purpose
Groq	POST /openai/v1/chat/completions (OpenAI-compatible)	Your backend's categorization engine, only for the unmatched long tail after the rules engine has run, and for generating plain-language insurance explanations
Supabase	Auth REST endpoints, via SDK	Consumer sign-up, login, and session refresh — not used by the lender-facing surface

13.6 The One Rule Worth Repeating

Every consumer endpoint must scope its query by the `user_id` taken from the verified session, never from a client-supplied parameter, and every lender endpoint must check a live `consent_grant` before returning anything about a specific business. Those two checks, applied without exception, are what prevent the two most costly mistakes available in this design: one user reading another user's financial data, and a lender reading a business's data without permission. Everything else in this API surface is a convenience; those two rules are the ones that cannot be relaxed at any phase of the build.